

Contents

Beispiel-Bots — Erklärung für den Dozenten	1
RandomBot	1
StillstandBot	1
ChaserBot	2
FluchtBot	3
PowerBot	4
Gemeinsame Muster über alle Beispiel-Bots	5

Beispiel-Bots — Erklärung für den Dozenten

Detaillierte Erklärung der fünf fertigen Referenz-Bots unter `bots/examples/` (`src/main/kotlin/bots/examples/`). Diese Bots werden von Schülern nicht verändert — sie dienen als Testgegner (Sparring-Partner) und als Referenz, auf die man verweisen kann, ohne die Musterlösungen aus [loesungen.md](#) direkt zu zeigen.

Alle Bots implementieren `RobotBrain` und werden über `BotRegistry.kt` automatisch in die Bot-Auswahl der App eingebunden — es muss dafür nichts manuell registriert werden (anders als bei `teamXBots`).

RandomBot

```
class RandomBot(override val name: String = "RandomBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val direction = Direction.entries.random()
        return if (Random.nextBoolean()) {
            Action.Move(direction)
        } else {
            Action.Shoot(direction)
        }
    }
}
```

Was er macht: Wählt jeden Tick unabhängig voneinander eine zufällige Richtung (`Direction.entries.random()`) und eine zufällige Aktion (`Random.nextBoolean()` entscheidet Move vs. Shoot). Kein Bezug zu `sensors.self` oder `sensors.others` — der Bot “sieht” nichts, er würfelt nur.

Intention: Einfachste denkbare Baseline. Dient als unterster Maßstab: jeder von Schülern gebaute Bot sollte gegen `RandomBot` zuverlässig gewinnen, sonst stimmt etwas mit der eigenen Logik nicht. Auch nützlich als allererstes Testduell, weil er garantiert keine Exception wirft und sich immer “irgendwie” verhält.

Didaktischer Bezug: Entspricht fast 1:1 Story 1.1 (Musterlösung siehe [loesungen.md](#)), nur zusätzlich mit der Move/Shoot-Zufallsentscheidung kombiniert.

StillstandBot

```
class StillstandBot(
    override val name: String = "StillstandBot",
    private val shootDirection: Direction = Direction.EAST
) : RobotBrain {
```

```

    override fun decide(sensors: Sensors): Action {
        return Action.Shoot(shootDirection)
    }
}

```

Was er macht: Bewegt sich nie, schießt jeden Tick stur in dieselbe, im Konstruktor festgelegte Richtung (Default EAST). shootDirection ist ein optionaler Konstruktorparameter — im Turnier/Testduell wird i.d.R. der Default verwendet, aber man könnte in BotRegistry/Tests auch StillstandBot(shootDirection = Direction.NORTH) instanziiieren.

Intention: Kontrollierbarster mögliche Testgegner. Weil er sich nie bewegt, lässt sich exakt vorher-sagen, ob ein Schuss trifft — ideal, um zu prüfen, ob ein Schüler-Bot (a) überhaupt trifft, wenn er in Reihe/Spalte steht, und (b) einem Dauerfeuer ausweicht, wenn er selbst getroffen wird. Wird in Story 2.1 explizit als Testgegner genannt.

Didaktischer Bezug: Entspricht direkt Story 2.1.

ChaserBot

```

class ChaserBot(override val name: String = "ChaserBot") : RobotBrain {
    override fun decide(sensors: Sensors): Action {
        val target = findNearestEnemy(sensors) ?: return Action.Wait

        val dx = target.position.x - sensors.self.position.x
        val dy = target.position.y - sensors.self.position.y

        return when {
            dx == 0 && dy < 0 -> Action.Shoot(Direction.NORTH)
            dx == 0 && dy > 0 -> Action.Shoot(Direction.SOUTH)
            dy == 0 && dx > 0 -> Action.Shoot(Direction.EAST)
            dy == 0 && dx < 0 -> Action.Shoot(Direction.WEST)
            abs(dx) >= abs(dy) -> Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
            else -> Action.Move(if (dy > 0) Direction.SOUTH else Direction.NORTH)
        }
    }

    private fun findNearestEnemy(sensors: Sensors): RobotState? {
        if (sensors.others.isEmpty()) return null
        return sensors.others.minByOrNull { other ->
            abs(other.position.x - sensors.self.position.x) +
            abs(other.position.y - sensors.self.position.y)
        }
    }
}

```

Was er macht: - findNearestEnemy sucht per Manhattan-Distanz ($\text{abs}(dx) + \text{abs}(dy)$, keine Diagonalen auf dem Grid) den nächstgelegenen lebenden Gegner. minByOrNull statt minBy, weil sensors.others leer sein kann (letzter Gegner tot) — dann liefert die Funktion null, und decide() fängt das mit ?: return Action.Wait ab, statt eine Exception zu werfen. - dx/dy ist der Vektor vom eigenen Bot zum Ziel. Die ersten vier when-Zeilen prüfen exakte Ausrichtung ($dx == 0 \rightarrow$ gleiche Spalte, $dy == 0 \rightarrow$ gleiche Reihe) und schießen sofort in die passende Richtung, sobald ein Treffer möglich ist. - Ist keine der vier Zeilen erfüllt (Ziel liegt diagonal versetzt), wird stattdessen bewegt: $\text{abs}(dx) \geq \text{abs}(dy)$ entscheidet, ob zuerst die X- oder die Y-Achse angeglichen wird — die Achse mit dem größeren Ausschlag wird zuerst geschlossen. Das führt über mehrere Ticks zu

einer treppenartigen (“diagonalen”) Annäherung, bis eine der beiden Differenzen 0 erreicht und ab dann geschossen wird.

Intention: Zeigt die “Angriff”-Grundlogik aus Epic 2 (Sichtlinie prüfen -> schießen, sonst verfolgen) in einer sauberen, vollständigen Form. Aggressiv, aber ohne jede Rücksicht auf eigene Gesundheit — stirbt ggf. lieber angreifend, als zu fliehen.

Didaktischer Bezug: Ist die “exakte, bereits getestete Referenz” für die Stories 2.2/2.3 (siehe Verweis in loesungen.md). Wer die Musterlösung zu 2.2/2.3 mit der echten Implementierung vergleichen will, findet hier die robustere Variante (u.a. mit sauberer null-Behandlung statt !!).

FluchtBot

```
class FluchtBot(override val name: String = "FluchtBot") : RobotBrain {

    private companion object {
        const val FLEE_HEALTH_THRESHOLD = 20
    }

    override fun decide(sensors: Sensors): Action {
        val target = findNearestEnemy(sensors) ?: return Action.Wait

        return if (sensors.self.health < FLEE_HEALTH_THRESHOLD) {
            fleeFrom(sensors, target)
        } else {
            attack(sensors, target)
        }
    }

    // findNearestEnemy/attack: siehe ChaserBot, identische Implementierung

    private fun fleeFrom(sensors: Sensors, target: RobotState): Action {
        val self = sensors.self.position
        val currentDistance = abs(target.position.x - self.x) + abs(target.position.y - self.y)

        val bestDirection = Direction.entries.maxByOrNull { direction ->
            val moved = self.moved(direction)
            abs(target.position.x - moved.x) + abs(target.position.y - moved.y)
        }

        return if (bestDirection != null) {
            val moved = self.moved(bestDirection)
            val newDistance = abs(target.position.x - moved.x) + abs(target.position.y - moved.y)
            if (newDistance > currentDistance) Action.Move(bestDirection) else Action.Wait
        } else {
            Action.Wait
        }
    }
}
```

Was er macht: - attack/findNearestEnemy sind wortwörtlich dieselbe Logik wie in ChaserBot (bewusst dupliziert statt geteilt — siehe Hinweis unten). - Der eigentliche Unterschied ist fleeFrom: statt wie in der Musterlösung zu 1.3 nur die Achse mit dem größeren Ausschlag zu betrachten, testet FluchtBot **alle vier Richtungen** durch (Direction.entries.maxByOrNull { ... }).

Für jede Richtung wird simuliert, wo der Bot landen würde (`self.moved(direction)` — Hilfsfunktion aus `Position`, siehe `Models.kt:15`), und dann die resultierende Distanz zum Gegner berechnet. Gewählt wird die Richtung mit der **größten** resultierenden Distanz (`maxByOrNull`). - Das ist eine robustere Fluchtstrategie als die einfache “größere Achse zuerst”-Heuristik: sie berücksichtigt korrekt auch Fälle, in denen eine Bewegung entlang der “falschen” Achse den Abstand trotzdem stärker vergrößert (z.B. wenn der Bot bereits an einer Wand steht und in eine Achse gar nicht mehr ausweichen kann). - Sicherheitscheck am Ende: `if (newDistance > currentDistance) Action.Move(...) else Action.Wait`. Das fängt den Fall ab, dass der Bot bereits in einer Ecke eingeklemmt ist und *jede* mögliche Bewegung den Abstand verkleinern oder gleich halten würde (die Engine blockt Bewegungen aus dem Arena-Raster ohnehin ab, aber `fleeFrom` selbst weiß das nicht — hier wird stattdessen defensiv geprüft, ob sich die simulierte Distanz überhaupt verbessert, bevor bewegt wird). Ohne diese Prüfung könnte der Bot sich “in die Ecke fliehen” und dort feststecken, obwohl Stehenbleiben (`Wait`) gleich gut oder besser wäre.

Intention: Vollständige, robustere Variante von Story 1.3 kombiniert mit Angriffsverhalten aus Epic 2. Zeigt, wie man eine einfache Heuristik (Achse mit größerem Ausschlag) durch systematisches Durchprobieren aller Optionen (`maxByOrNull` über alle 4 Richtungen) ersetzen kann, wenn man eine nachweislich bessere Entscheidung treffen will.

Didaktischer Bezug: Referenz für Story 1.3. Für Schüler, die nach der Musterlösung fragen “warum reicht das nicht, wenn ich in eine Ecke laufe” — genau dieser Randfall wird hier durch die Vier-Richtungen-Prüfung sauber gelöst.

PowerBot

```
class PowerBot(override val name: String = "PowerBot") : RobotBrain {

    override fun decide(sensors: Sensors): Action {
        val self = sensors.self
        val enemies = sensors.others
        if (enemies.isEmpty()) return Action.Wait

        val alignedEnemies = enemies.filter { isAligned(self.position, it.position) }
        if (alignedEnemies.isNotEmpty()) {
            val weakest = alignedEnemies.minWithOrNull(
                compareBy<RobotState> { it.health }.thenBy { manhattanDistance(self.position, it.position) }
            )!!
            return Action.Shoot(directionTo(self.position, weakest.position))
        }

        val target = bestTarget(self.position, enemies)
        return moveTowardAlignment(self.position, target.position)
    }

    private fun bestTarget(self: Position, enemies: List<RobotState>): RobotState =
        enemies.minBy { manhattanDistance(self, it.position) + it.health / 10 }

    private fun moveTowardAlignment(self: Position, target: Position): Action {
        val dx = target.x - self.x
        val dy = target.y - self.y
        return if (dx != 0) {
            Action.Move(if (dx > 0) Direction.EAST else Direction.WEST)
        } else {
            Action.Wait
        }
    }
}
```

```

        Action.Move(if (dy > 0) Direction.SOUTH else Direction.NORTH)
    }
}
}

```

Was er macht — Schritt für Schritt: 1. **Sichtlinie zuerst prüfen, aber mit Zielpriorisierung:** `alignedEnemies` filtert alle Gegner, die bereits in Reihe/Spalte stehen (`isAligned`). Stehen mehrere davon in Sichtlinie, wird nicht einfach der nächste genommen (wie bei `ChaserBot`), sondern per `compareBy<RobotState> { it.health }.thenBy { manhattanDistance(...) }` sortiert: primäres Kriterium ist die niedrigste HP (schwächster zuerst töten), erst bei Gleichstand entscheidet die Distanz. `minWithOrNull` mit einem zusammengesetzten Comparator ist der Kotlin-Weg, nach mehreren Kriterien in Prioritätsreihenfolge zu sortieren — praktisch die direkte Umsetzung der Musterlösung zu Story 3.2 (Zielpriorisierung), nur in Kombination mit Sichtlinien-Filterung. 2. **Kein Gegner in Sichtlinie -> Ziel per Score wählen:** `bestTarget` berechnet für jeden Gegner einen kombinierten Score aus Distanz und Gesundheit (`manhattanDistance(...) + it.health / 10`) und wählt das Minimum — nahe UND schwache Gegner werden bevorzugt, nicht nur die kleinste Distanz wie bei `ChaserBot`. Die Gewichtung `health / 10` ist eine bewusst gewählte Konstante (bei `startHealth = 100` reicht die Health-Spanne 0..100, geteilt durch 10 also 0..10 — vergleichbar mit typischen Distanzwerten auf einem 10×10-Feld), keine “offizielle” Formel, sondern ein empirisch funktionierender Kompromiss. 3. **Ausrichten:** `moveTowardAlignment` schließt zuerst die X-Differenz (peilt dieselbe Spalte an), erst wenn `dx == 0`, wird die Y-Achse angeglichen. Anders als bei `ChaserBot` (größere Achse zuerst) ist hier die Reihenfolge fest “X vor Y” — laut Doc-Kommentar im Quellcode empirisch als stärkste Annäherungsstrategie gegen die anderen Beispiel-Bots getestet. 4. **Bewusst keine Flucht.** Anders als `FluchtBot` hat `PowerBot` keinen Health-Schwellenwert, unter dem er flieht. Das ist eine explizite Design-Entscheidung, keine vergessene Story: gegen einen Gegner mit demselben Fluchtmuster (z.B. `FluchtBot` selbst) würde ein Flucht-Trigger dazu führen, dass sich beide Bots synchron im Kreis jagen, ohne sich je zu treffen -> Unentschieden statt Sieg. Aggressiv bleiben ist hier die bessere Strategie.

Bekannte Grenze (kein Bug): Gegen einen exakt gleich starken, ähnlich aggressiven Bot (gleiche Start-HP, gleicher Schaden pro Treffer, z.B. `ChaserBot`) endet der Kampf zwangsläufig unentschieden — stehen beide in derselben Linie und feuern gleichzeitig, sterben sie bei `startHealth = 100` und `damagePerHit = 10` nach exakt 10 gegenseitigen Treffern gleichzeitig. Das ist schlicht Arithmetik der Engine-Defaults, keine Schwäche der Zielwahl-Logik.

Intention: Der aggressivste und “klügste” Beispiel-Bot — kombiniert Zielpriorisierung (Epic 3) mit einer verfeinerten Bewegungsstrategie. Dient als starker Endgegner/Benchmark: Schüler-Teams, deren Bot regelmäßig gegen `PowerBot` gewinnt, haben vermutlich eine sehr solide Strategie gebaut.

Didaktischer Bezug: Gute Referenz für Story 3.2 (Zielpriorisierung) in einer fortgeschritteneren Form (Score aus zwei Kriterien statt nur einem). Auch geeignet, um im Review mit stärkeren Teams über Grenzen von Heuristiken zu sprechen (z.B. die “Gleichstand endet unentschieden”-Eigenschaft als Diskussionsanlass für Story 3.3, die Kür-Aufgabe).

Gemeinsame Muster über alle Beispiel-Bots

- **Manhattan-Distanz** (`abs(dx) + abs(dy)`) statt euklidischer Distanz — passend zum Grid, auf dem nur orthogonale Bewegungen möglich sind. Taucht in jedem Bot außer `RandomBot/StillstandBot` auf.
- **`minByOrNull/null`-Behandlung statt !!** bei der Gegnersuche (außer an einer Stelle in `PowerBot`, wo die Nicht-Leerheit vorher durch `alignedEnemies.isNotEmpty()` sichergestellt ist — dort ist !! sicher).
- **Y-Achse wächst nach unten** (Bildschirmkoordinaten, siehe Kommentar in `Models.kt:6`): NORTH bedeutet `y - 1`, nicht `y + 1`. Häufigste Verwechslungsquelle bei Schülern, die eigene

Richtungslogik schreiben — beim Review gegenprüfen.

- **Keine Bots werfen Exceptions bei leerem sensors . others** (alle haben einen frühen return Action.Wait-Pfad) — das ist auch ein explizites Akzeptanzkriterium in mehreren Backlog-Stories (z.B. 1.3) und sollte bei Schüler-Code genauso eingefordert werden.